System Architecture and Integration: Technical Documentation

Final Project: Containerized Infrastructure and CI/CD Automation using Docker and Jenkins

Project Name: PlaylistHub

1. Team Composition and Roles

- **Course / Section:** ITE-303

Name	Role	Primary Deliverables
Aquino, Kurt Justin	**Project Lead / Scrum Master**	Repository structure, project proposal, demo coordinator, task management
Sy, Jim Ellen Faith	**DevOps / CI-CD Engineer**	Jenkins containerization, credentials, Jenkinsfile, ngrok webhook
Salao, Ronin Charles	**Infrastructure Engineer**	Dockerfiles, docker-compose.yml, reverse proxy, networks & volumes
Gatus, Nicole Ann & Santoyo, Rance Jyle	**Backend and Database Engineer**	FastAPI & Express modules, PostgreSQL schema, seed data, unit test suites
Enriquez, Julius	**Frontend, QA, and Documentation Lead**	Web UI, smoke tests, technical documentation, slide deck

2. Project Description and Module Breakdown

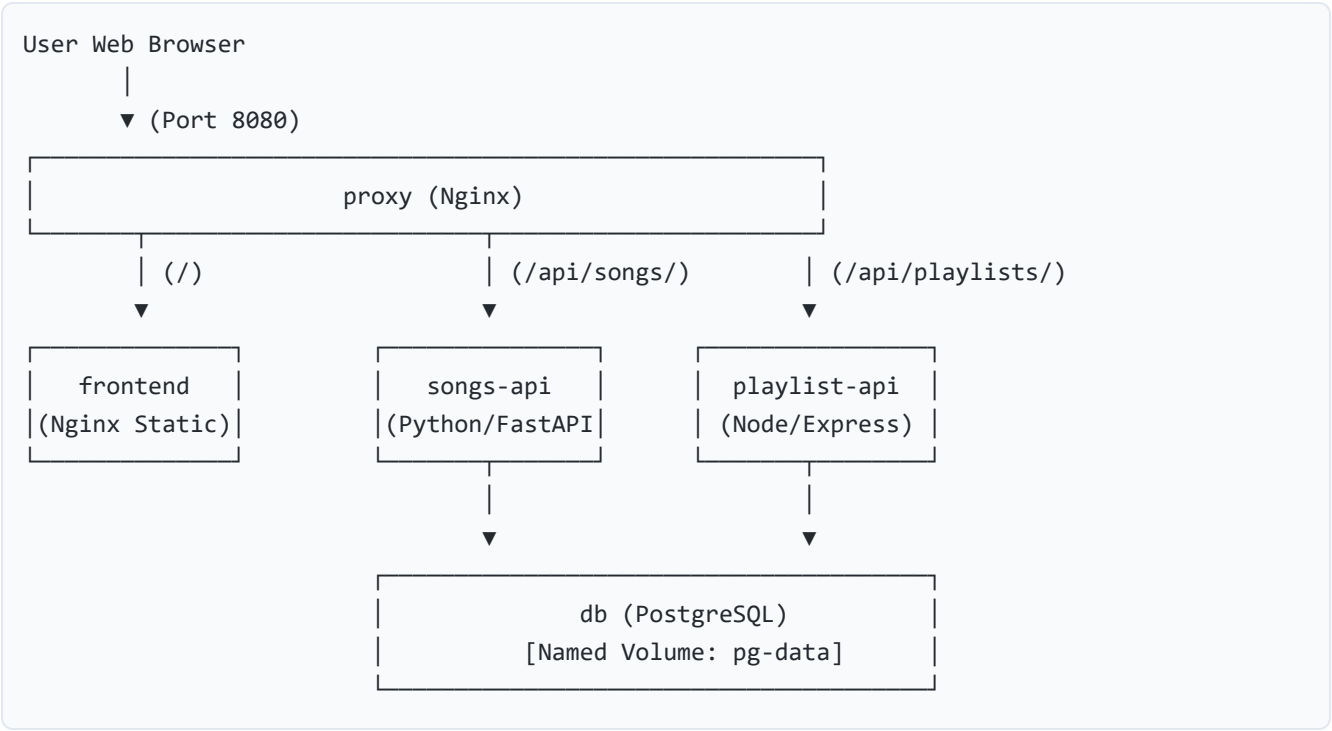
PlaylistHub is a containerized microservice-based music playlist application built to demonstrate production-grade CI/CD automation and container orchestration.

Application Modules & Roles

1. `proxy` **(Reverse Proxy):** Single external gateway listening on port 8080. Dispatches incoming HTTP traffic:
2. `/` to `frontend`
3. `/api/songs/` to `songs-api`
4. `/api/playlists/` to `playlist-api`
5. `frontend` **(Web Interface):** Lightweight, responsive static web client served via Nginx. Displays song catalogs, manages playlist additions, and features an unmistakable release badge (`Build #X`).
6. `songs-api` **(Song Catalog Service):** REST API written in Python FastAPI. Manages song metadata (title, artist, genre, duration) with strict validation.
7. `playlist-api` **(Playlist Service):** REST API written in Node.js Express. Manages custom playlists, track associations, and computes total playlist duration.
8. `db` **(Relational Database):** PostgreSQL 16 container storing song records and playlist relationships. Backed by a named Docker volume (`pg-data`).

---

3. Architecture & Request Flow



Network Topology & Port Isolation

- **`frontend-net`:** Connects `proxy`, `frontend`, `songs-api`, and `playlist-api`.
- **`backend-net`:** Connects `songs-api`, `playlist-api`, and `db`.
- **Port Security:** Only the `proxy` publishes a port (`8080:80`) to the host. The PostgreSQL container (`5432`) is strictly isolated inside `backend-net`, preventing any unauthorized external access.

---

4. Technology Stack Justification

Technology	Selection	Justification
Reverse Proxy	Nginx 1.27-Alpine	Ultra-lightweight, industry standard, fast static reverse proxy with low memory footprint.
Frontend	Static HTML5/CSS3/Vanilla JS in Nginx	Avoids heavy Node/bundler overhead, instant image builds, minimal layer size, zero build failure risk during live demos.
Service A	Python 3.12 + FastAPI	Native asynchronous performance, automatic Pydantic validation, concise code, rich ecosystem for data operations.
Service B	Node.js 20 + Express	Demonstrates a polyglot microservice environment alongside Python; event-driven I/O is optimal for lightweight API operations.
Database	PostgreSQL 16 (Alpine)	ACID-compliant relational DB with foreign key cascades for relational integrity (`playlist_songs` junction table).
CI/CD	Jenkins LTS	Robust declarative pipeline syntax, native Docker socket integration, standard pipeline-as-code support.
Testing	PyTest + Jest	High-speed unit testing frameworks integrated directly into Dockerfile multi-stage test targets.

---

5. Dockerfile Design & Security Choices

All application images implement best practices: 1. **Pinned, Slim Base Images:** E.g., `python:3.12-slim`, `node:20-alpine`, `nginx:1.27-alpine` to reduce attack surface and build size. 2. **Layer Caching:** Dependency manifests (`requirements.txt`, `package.json`) are copied and installed before application code. Source changes do not re-trigger dependency downloads. 3. **Multi-Stage Builds with Test Targets:** - `test` target builds dependency caches, runs test suites inside Docker, and ensures the host

machine requires no local Node or Python runtimes. - `production` target strips development dependencies (`--omit=dev`). 4. **Least-Privilege Non-Root Execution:** - `songs-api` runs under unprivileged user `appuser` (`uid 10001`). - `playlist-api` runs under unprivileged system user `node` . 5. **Baked Configuration:** Initialization SQL and Nginx configurations are copied directly into images, eliminating file-permission issues across host volume mounts. 6. **Healthchecks:** Each container implements explicit `HEALTHCHECK` instructions using HTTP ping endpoints (`/health` or `/nginx-health`).

---

6. Docker Compose Orchestration

The `docker-compose.yml` provides a single-command deployment: - **Dependency Handling:** Services declare `depends_on` conditioned on `condition: service_healthy` , ensuring the database accepts connections before APIs boot, and APIs respond before the proxy binds. - **Named Volume Persistence:** The PostgreSQL directory `/var/lib/postgresql/data` maps to the named volume `pg-data` . - **Environment Management:** Sensitive database credentials pass via variables (`${DB_USER}` , `${DB_PASSWORD}`) with fallback defaults for local development.

---

7. Jenkins Setup & Configuration

- **Isolated Jenkins Container:** Jenkins runs in `infra/jenkins/` with a mounted `/var/run/docker.sock` and root permissions (lab environment configuration), allowing it to control the host Docker daemon.
- **Persistent State:** Jenkins configuration and job history reside in the named volume `jenkins_home` .
- **Credentials Vault:** Production secrets are stored under `Manage Jenkins -> Credentials` as a Secret File with ID `playlisthub-env` .
- **Dual Triggers:**
- *Webhook:* Configured via ngrok tunnel (`https://<ngrok-id>/github-webhook/`).
- *Poll SCM:* Configured with `pollSCM('H/2 * * * *')` as an automatic fallback.

---

8. Pipeline Explanation (Stage by Stage)

```
[ Checkout ] ──► [ Test (Parallel) ] ──► [ Build Images ] ──► [ Deploy ] ──► [ Smoke Test ]
                        │
             (If tests fail, pipeline aborts here)
```

1. **Checkout:** Clones the latest commit from the Git repository.
2. **Test (Quality Gate):** Concurrently triggers `docker build --target test` for both `songs-api` and `playlist-api`. If any assertion fails, the build halts immediately. Broken code never reaches production.
3. **Build Images:** Executes `docker compose build`, baking the Jenkins `BUILD_NUMBER` tag into the image binaries and frontend index page.
4. **Deploy:** Uses Jenkins `withCredentials` to write out the `.env` file securely, then invokes `docker compose up -d --no-build --remove-orphans`.
5. **Smoke Test:** Executes 15 retry probes querying `/api/songs/health`, `/api/playlists/health`, and verifying the frontend HTML contains `Build #${TAG}`.
6. **Post Actions:** Always deletes the temporary `.env` file to prevent secret leakage.

---

9. Testing & Quality Gate Strategy

- **`songs-api` Tests:**
- `test_health_endpoint`: Asserts HTTP 200 and service identification.
- `test_song_model_validation_*`: Verifies Pydantic rejects invalid inputs (negative duration, empty titles).
- **`playlist-api` Tests:**
- `duration.test.js`: Verifies duration summation (`computeTotalDuration`) and string formatting (`formatDuration`).
- `app.test.js`: Verifies route logic and HTTP status codes using Supertest.

Quality Gate Enforcement: If a developer commits broken code (e.g., faulty duration calculation), the test step in Jenkins fails with a non-zero exit code. Docker stops the pipeline prior to the Deploy stage, leaving the running system completely unharmed.

---

10. Rollback Procedure & Data Persistence Proof

Data Persistence

PostgreSQL state is retained across container lifecycles in the `pg-data` named volume. Stopping and recreating containers with `docker compose down` followed by `docker compose up -d` preserves all previously inserted records.

Automated Rollback Procedure

1. Jenkins contains a companion pipeline job: `playlisthub-rollback` (defined in `Jenkinsfile.rollback`).
2. Input parameter `TARGET_TAG` specifies the desired previous build number (e.g., `1`).
3. The job verifies that the target Docker image exists, writes credentials, redeploys without building (`--no-build`), and verifies the live web page reflects the rolled-back build number.

---

11. Problems Encountered and Solutions

1. **Host-to-Container Secret Leakage:**
2. *Problem:* Hardcoding database credentials in Git commits creates serious security vulnerabilities.
3. *Solution:* Stored credentials in Jenkins Credentials store as a Secret File, dynamically injected during the Deploy stage and deleted in the `always` post-block.
4. **Windows CRLF Line Ending Incompatibilities:**
5. *Problem:* Git on Windows checked out shell scripts and Dockerfiles with `\r\n`, breaking Linux container execution.
6. *Solution:* Added `.gitattributes` with `* text=auto eol=lf` to enforce Unix line endings repository-wide.
7. **Database Race Conditions on Startup:**
8. *Problem:* APIs attempted to connect before PostgreSQL finished initialization.
9. *Solution:* Configured Docker Compose `healthcheck` on the database with `pg_isready`, and coupled it to API definitions with `depends_on: { db: { condition: service_healthy } }`.

---

12. Individual Contributions

Member	Role	Main files owned	Key Contributions
Aquino, Kurt Justin	Project Lead / Scrum Master	`README.md`, `docs/project-proposal.md`, GitHub PR coordination	Project scaffolding, governance, workflow integration, final merge reviews
Sy, Jim Ellen Faith	DevOps / CI-CD Engineer	`infra/jenkins/`, `Jenkinsfile`, `Jenkinsfile.rollback`	Jenkins container setup, Docker socket integration, declarative pipeline, rollback job
Salao, Ronin Charles	Infrastructure Engineer	`docker-compose.yml`, `proxy/`, Dockerfiles, `docs/architecture.png`	Container orchestration, reverse proxy routing, network isolation, architecture visual
Gatus, Nicole Ann	Backend and Database Engineer	`songs-api/`, `playlist-api/`, `db/`	PostgreSQL schema, FastAPI catalog service, Express playlist service
Santoyo, Rance Jyle	Backend and Database Engineer	`songs-api/`, `playlist-api/`	409 conflict duplicate handling, song update PUT endpoint, unit test enhancements
Enriquez, Julius	Frontend, QA, and Documentation Lead	`frontend/`, `scripts/`, technical report, slides	Responsive web dashboard, modal playlist picker, smoke test automation, slide deck

Detailed per-role task lists and review preparation are in `docs/team-contribution-guide.md`.

---

13. References

- Official Docker Documentation: https://docs.docker.com/
- Jenkins Declarative Pipeline Guide: https://www.jenkins.io/doc/book/pipeline/
- FastAPI Documentation: https://fastapi.tiangolo.com/
- PostgreSQL Docker Official Image: https://hub.docker.com/_/postgres